

Day 13 — full EDA portfolio project

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · study lesson with explanations, paired with day13_eda_portfolio.py

1. Why this day is different

No new concepts today. Days 1-12 taught cleaning, feature engineering, statistics, and SQL as separate skills. Today they get combined into one pipeline that runs start to finish on the same dataset — this is the artifact that goes on a resume and gets discussed in an interview, not any single day's script on its own.

2. Load, clean, and engineer features once

Every later step (statistics, SQL, the dashboard) depends on the same clean DataFrame. The project does this exactly once, in one function, so nothing gets cleaned twice or cleaned differently in different places.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df = df.drop(columns=["Cabin"])
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df["AgeGroup"] = pd.cut(df["Age"], bins=[0, 12, 18, 35, 60, 100],
                        labels=["Child", "Teen", "Adult", "MiddleAge", "Senior"])
df["Title"] = df["Name"].str.extract(r",\s*(^[.]+)\.")
```

- `fillna(median())` replaces missing ages with the middle value — robust to the outliers Day 10 found
- `pd.cut()` buckets a continuous column into labeled ranges — this is Python doing what Day 12's SQL CASE WHEN did, on the DataFrame side
- `str.extract()` pulls a regex capture group out of the Name column — grabs the title (Mr, Mrs, Miss...) between the comma and the period

3. Statistical summary, tied to plain-English findings

Day 10 taught t-tests and chi-square tests as concepts. Here, every test result gets converted into one sentence a non-technical reader could understand — that sentence is what actually goes in the README, not the raw p-value.

```

fare_survived = df.loc[df["Survived"] == 1, "Fare"]
fare_died = df.loc[df["Survived"] == 0, "Fare"]
t_stat, t_p = stats.ttest_ind(fare_survived, fare_died, equal_var=False)

finding = (
    f"Survivors paid ${fare_survived.mean():.2f} avg fare vs "
    f"${fare_died.mean():.2f} for non-survivors (t={t_stat:.2f}, p={t_p:.2e})"
)

```

- `equal_var=False` runs Welch's t-test instead of Student's — safer default, doesn't assume both groups have equal variance
- The f-string builds a human-readable sentence directly from the test's numbers, so the statistical result and its plain-English meaning never drift apart

4. SQL summary tables, cross-checking the same findings

The cleaned DataFrame gets loaded into an in-memory SQLite database

(`sqlite3.connect(":memory:")`) so the same survival patterns can be reproduced in SQL — not because SQL is required here, but because a portfolio project that shows both pandas and SQL solving the same problem is stronger than one that only shows one.

```

conn = sqlite3.connect(":memory:")
df.to_sql("titanic", conn, index=False)

pd.read_sql("""
    SELECT Pclass, Sex, COUNT(*) AS n_passengers,
           ROUND(AVG(Survived), 3) AS survival_rate
    FROM titanic
    GROUP BY Pclass, Sex;
""", conn)

```

- `:memory:` creates a database that lives only in RAM — no .db file left behind, perfect for a throwaway cross-check
- `df.to_sql(...)` writes the whole DataFrame into that database as a real table in one line
- Also included: CASE WHEN fare-tier bucketing and a RANK() window function for top fares per class — the same tools from Day 12, reused here

5. The dashboard: one image, six panels

This is the single image that represents the whole analysis. Built with `plt.subplots(2, 3, ...)` — a grid of 6 charts sharing one figure, saved as one PNG instead of six separate files.

```
fig, axes = plt.subplots(2, 3, figsize=(16, 9))

axes[0, 0].bar(["Died", "Survived"], survived_counts.values)
axes[0, 1].bar(sex_survival.index, sex_survival.values)
axes[0, 2].bar(class_survival.index.astype(str), class_survival.values)
axes[1, 0].hist(df["Age"], bins=30)
axes[1, 1].hist(np.log1p(df["Fare"]), bins=30)
sns.heatmap(corr, annot=True, ax=axes[1, 2])
```

- `axes[row, col]` addresses one specific panel in the grid — each line targets exactly one chart
- Fare uses `np.log1p()` before plotting because Day 10 found it heavily right-skewed — the log transform makes the histogram actually readable
- `sns.heatmap()` reuses the same correlation matrix computed in the statistics step, drawn directly into the last panel

6. Auto-generating the README

The findings computed in step 3 get written straight into a Markdown file — the write-up isn't typed by hand after the fact, it's generated from the same numbers the analysis produced, so the README can never say something the data doesn't actually show.

```
findings_md = "\n".join(f"- {f}" for f in stats_results["findings"])

content = f"""# Titanic EDA – full portfolio project
...
## Key findings

{findings_md}
"""

with open("README.md", "w", encoding="utf-8") as f:
    f.write(content)
```

- The triple-quoted f-string is a template — everything in `{ }` gets swapped for real values when the string is built
- `"\n".join(...)` turns the list of finding sentences into one block of Markdown bullet points, one per line

7. Why this order matters

`main()` calls these functions in a specific sequence: clean first, then statistics, then SQL, then the dashboard, then the README last. Every later step consumes something an earlier step produced —

the dashboard needs the correlation matrix computed in step 3, and the README needs the finding sentences from that same step. Nothing after step 2 ever touches the raw, uncleaned data again.

8. Deliverable and push

Three files make up this deliverable: the script itself, the dashboard PNG it generates, and the README it writes.

```
git add day13_eda_portfolio.py day13_dashboard.png README.md
git commit -m "Day 13: Full EDA portfolio project - cleaning, stats, SQL, and dashboard"
git push
```